Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Обработка разреженных матриц

Вариант № 9

Студент: **Дудырев Д.С.**

Группа: **ИУ7-32Б**

Преподаватель: **Барышникова М.Ю.**

2025 г.

Отчет лаб. работа №3

Описание условий задачи

Разработать программу умножения или сложения разреженных матриц. Предусмотреть возможность ввода данных, как с клавиатуры, так и использования заранее подготовленных данных. Матрицы хранятся и выводятся в форме трех объектов. Для небольших матриц можно дополнительно вывести матрицу в виде матрицы. Величина матриц - любая. Сравнить эффективность (по памяти и по времени выполнения) стандартных алгоритмов обработки матриц с алгоритмами обработки разреженных матриц при различной степени разреженности матриц и различной размерности матриц.

Техническое задание

Исходные данные

txt файл, содержащий две матрицы. Перед каждой матрицой указываются размер матрицы, далее идут элементы матрицы в двух вариантах: классическая запись, координатная запись.

Пользовательский ввод. Сначала для каждой матрицы предлагается ввести размеры, далее ввод элементов. После выполнения успешного умножения матриц идет ввод типа вывода: классический, координатный, CSR.

Выходные данные

Результат умножение матриц, выводиться одним из трех способов: классический вывод, координатный вывод, в виде CSR данных. Результат выполнения сравнения умножения матриц представленных в классическом и CSR/CSC форматах

Описание реализуемой задачи

Ввод матриц двумя способами: классическим и координатным. Перенос матриц из формата массив массивов чисел в форматы CSR, CSC. Реализовать выполнение умножение матрицы CSR на CSC. Вывод матрицы в классическом формате и координатном.

Способы обращения к программе

Программа запускается из командной строки при помощи команды `app.exe`.

Возможные аварийные ситуации и ошибки пользователя

Аварийная ситуация	Код ошибки	Сообщение
Недопустимый ввод	10	
Введен отрицательный размер матрицы	11	
При координатном вводе индекс превышает размер матрицы	13	
Количество элементов превышает максимальное количество в матрице	14	
Ввод команды больше максимальной или отрицательной	15	
Длина пути к файлу больше 250	16	
Ошибка открытия файла	17	
Размер матрицы 0	18	

Аварийная ситуация	Код ошибки	Сообщение
Матрицы не перемножаемые	19	
Ошибка выделения памяти	20	

Описание структуры данных

Хранение структур в программе

Для хранения изначальных матриц использует динамическое выделение памяти, которое выполняется после чтения размеров матрицы из файла или ввода. Для структур CSR и CSC память выделяется автоматически при преобразовании обычной матрицы в указанным типам.

Структуры

Структура для хранения данных в виде CSR или CSC.

```
typedef struct
{
    type_matrixt type;

    size_m n;
    size_m m;

    unsigned int max_values;

    int *values;
    size_m *ind;
    size_m *dim;
} matrix_t;
```

- `type` – тип хранения CSR/CSC
- `n` – кол-во строк в матрице
- `m` – кол-во столбцов в матрице
- `max_values` – макисмальное кол-во ненулевых элементов
- `values` – ненулевые элемент
- `ind` – индексы значений по основному измерению
- `dim` – количество ненулевых чисел до i-й второстепенного измерения включительно
  Для CSR основным измерением является строка, второстепенным столбцы.
  Для CSC основным измерением является столбцы, второстепенным строки.

Структура для хранения значений основного измерения матриц CSR или CSC.

```
typedef struct
{
    size_m *ind;
    int *value;
    size_m len;
} dim_data_t;
```

- `ind` – индекс элемента
- `dim` – значение элемента
- `len` – количество элементов

Классическая матрица

Для хранения матрицы используется двойной указатель на тип данных `**int`

Описание алгоритма

Умножение CSR на CSC

```
m_csr — матрица записаная в формате CSR
m_csr_res — результат умно;ения в виде CSR
m_csc — матрица записаная в формате CSC
get_row(m_csr, i) — получить i-ю строку из матрицы m_csr
get_col(m_csc, i) — получить i-й столбец из матрицы m_csc
mulr_row_col(row, col) — результат умножения строки на столбце

num_count = 0
Для i от 0 до m_csr.n
    Для i от 0 до m_csc.c
        row = get_row(m_csr, i)
        col = ger_col(m_csc, j)
        val = mulr_row_col(row, col)

        Если val == 0
            Продолжить
        Конец Если

        m_csr_res.values[num_count] = val
        m_csr_res.ind[num_count] = j
        num_count = num_count + 1
    Конец Для
    m_csr_res.dim[i] = num_count
Конец Для
```

Перенос матрицы в CSR

```
m_csr — матрица в представлении CSR
mtr — исходная матрица
n — кол-во строк
m — кол-во столбцов

num_count = 0
Для i от 0 до n
    Для j от 0 до m
        Если mtr[i][j] == 0
            Продолжить
        Конец Если
            m_csr.values[num_count] = mtr[i][j]
            m_csr.ind[num_count] = j
            num_count = num_count + 1
        Конец Если
        m_csr.dim[i] = num_count
```

```
    Конец Для
Конец Для
```

Перенос матрицы в CSC

```
m_csc — матрица в представлении CSC
mtr — исходная матрица
n — кол-во строк
m — кол-во столбцов

num_count = 0
Для i от 0 до n
    Для j от 0 до m
        Если mtr[j][i] == 0
            Продолжить
        Конец Если
            m_csr.values[num_count] = mtr[j][i]
            m_csr.ind[num_count] = j
            num_count = num_count + 1
        Конец Если
            m_csr.dim[i] = num_count
        Конец Для
Конец Для
```

Необходимые функции

`typedef int (*method_input_matrix)(matrix_data_t matrix, size_m n, size_m m, FILE *f);`

Ввод

`int input_size(size_m *num, FILE *f)` – ввод размерности

- num – указатель на вводимую размерность
- f – файл для считывания данных
  `int input_num(int *num, FILE *f)` – ввод целого числа
- num – указатель на вводимое число
- f – файл для считывания данных
  `int input_str(char *str, size_t max_len, FILE *f)` – ввод строки
- str – вводимая строка
- max_len – максимальная длина вводимой строки
- f – файл для считывания данных
  `int input_matrix(matrix_data_t *matrix, size_m *n, size_m *m, FILE *f, method_input_matrix input_func)` – ввод матрицы
- matrix – вводимая матрица
- n – указатель на кол-во строк
- m – кол-во колонок
- f – файл для считывания данных
- input_func – метод ввода матрицы
  `int input_matrix_sizes(size_m *n, size_m *m, FILE *f)` – ввод размерностей для матрицы
- n – кол-во строк
- m – кол-во колонок

- f – файл для считывания данных
  `int input_matrix_classic(matrix_data_t matrix, size_m n, size_m m, FILE *f)` – ввод матрицы классическим способом
- matrix – Вводимая матрица
- n – кол-во строк
- m – кол-во колонок
- f – файл для считывания данных
  `int input_matrix_coord(matrix_data_t matrix, size_m n, size_m m, FILE *f)` – ввод матрицы координатным способом
- matrix – Вводимая матрица
- n – кол-во строк
- m – кол-во колонок
- f – файл для считывания данных
  `int input_command(int *cmd, command_t type)` – ввод команды
- cmd – значение команды
- type – тип команды

Вывод

`void print_matrix(matrix_data_t matrix, size_t n, size_t m);` – вывод матрицы в классическом виде

- matrix – матрица
- n – кол-во строк
- m – кол-во столбцов
  `void print_csr_matrix(matrix_t *mtr);` – вывод матрицы CSR в классическом виде
- mtr – матрица в формате CSR
  `void print_csr_coord(matrix_t *mtr);` – вывод матрицы CSR в координатном виде
- mtr – матрица в формате CSR

Обработка матриц

`int matrix_to_csr(matrix_t *m_csr, matrix_data_t mtr, size_m n, size_m m);` – перенос обычной матрицы в формат CSR

- m_csr – разряженная матрица в формате CSR
- mtr – обычная матрица
- n – кол-во строк
- m – кол-во столбцов
  `int matrix_to_csc(matrix_t *m_csr, matrix_data_t mtr, size_m n, size_m m);`
- m_csc – разряженная матрица в формате CSC
- mtr – обычная матрица
- n - кол-во строк
- m – кол-во столбцов
  `int mult_row_by_col(dim_data_t *d_csr, dim_data_t *d_csc);`
- d_csr – данные о значениях и их индексах в строке
- s_csc – данные о значениях и их индексах в столбце
  `int mult_csr_by_csc(matrix_t *mtr, matrix_t *m_csr, matrix_t *m_csc);`
- mtr – Результат умножения в формате CSR

- m_csr – разряженная матрица в формате CSR
- m_csc – разряженная матрица в формате CSC
  `int classic_mult(matrix_data_t *res, matrix_data_t a, matrix_data_t b, size_t n, size_t m, size_t k);` – классическое умножение матрицы, строка на столбец
- res – матрица, результат умножения
- a – Левая матрица
- b – Правая матрица
- n – кол-во строк в матрице **a**
- m – кол-во столбцов в **a** и кол-во строк в **b**
- k – кол-во столбцов в **b**

Тесты

Негативные

Ввод	**Вывод**	**Код ошибки**
Матрица_1 NxM Матрица_2 KxP	Матрицы не умножаются	19
Ввод отрицательной размерности матрицы	Отрицательная размерность	11
Ввод нулевой размерности матрицы	Размерность нулевая	18
Ввод символа для размерности	Некоректный ввод	10
Ввод символа для элемента матрицы	Некоректный ввод	10
Ввод координат больших чем размермы матрицы	Индекс вне диапазона матрицы	13
Ввод отрицательных координат	Некоректный ввод	10
Вводиться не числом при вводе матрицы коодринаным способов	Некоректный ввод	10
Ввод не существующего файла	Ошибка открытия файла	17
Путь к файлу больше 250 символов	Слишком длинная строка	16
Номер вводимой команды больше допустимого	Неизвестный номер команды	15
Номер вводимой команды отрицательный	Неизвестный номер команды	15
Количество элементов при коодринатном вводе больше максимального количества элементов в матрице	Указано слишком много элеменов для матрицы	14

Позитивные

Ввод	**Вывод**	**Комментарий**
Матрица_1 MxN Нулевая_матрица_2 NxN	Нулевая_матрица MxN	Умножение на нулевую матрицу
Матрица_1 MxN Единичная_матрица_2 NxN	Матрица_1	Умножение на единичную матрицу
Матрица_1 3x3 Матрица_2 3x3	Матрица_3 3x3	Умножение квадратных матриц
Матрица_1 4x3 Матрица_2 3x4	Матрица_3 4x4	Умножение матрицы с расширением

Ввод	Вывод	Комментарий
Матрица_1 4x3 Матрица_2 3x2	Матрица_3 4x2	Умножение матрицы с уменьшением
Матрица_1 4x1 Матрица_2 1x4	Матрица_3 4x4	Умножение матрицы стоблца на матрицу строку
Матрица_1 1x4 Матрица_2 4x1	Матрица_3 1x1	Умножение матрицы строки на матрицу столбец
Матрица_1 1x1 Матрица_2 1x1	Матрица_3 1x1	Умножение единичных матриц
Матрица_1 30x30 Матрица_2 30x30 Координатный ввод	Матрица_3 30x30	Умножение больших матриц

Классический ввод

```
Ввод матрицы A
Необходимо ввести матрицу полностью
Введите n: 3
Введите m: 3
Вводите матрицу:
1 2 3
1 2 3
1 2 3

Ввод матрицы B
Необходимо ввести матрицу полностью
Введите n: 3
Введите m: 3
Вводите матрицу:
4 5 6
1 2 3
4 5 6

Тип вывода:
1. Матричный вывод
2. Координатный вывод
3. CSR файл
0. Выход
Введите номер команды: 1
   18   24   30
   18   24   30
   18   24   30
```

Координатный ввод

```
Ввод матрицы A
Ввод матриц в формате: i j v
i — Номер столбца
j — Номер строки
v — Значение элемента
Введите n: 100
Введите m: 100
50 50 100


Ввод матрицы B
Ввод матриц в формате: i j v
i — Номер столбца
j — Номер строки
v — Значение элемента
Введите n: 100
Введите m: 100
50 50 2


Тип вывода:
1. Матричный вывод
2. Координатный вывод
3. CSR файл
0. Выход
Введите номер команды: 2

Вывод в формате i j v
i — строка
j — колонка
v — значение
50 50 200
Введите номер команды: 3
Индекс в строке:  49
       Значение: 200
кол-во в строке:   0   0   0
```

Количество элементов	Максимальная заполненость, %	
	Времени	Памяти
50	-	-
100	10	50
400	10	50
1000	10	30
4000	10	75
5000	10	50
10.000	10	50
500.000	10	30

Выводы

На основе полученных данных можно делать следующие выводы. Для матриц до 50 элементов использование CSR/CSC не дает преимуществ по используемой памяти или времени выполнения. При большем количестве элементов, использование CSR/CSC дает преимущество по скорости выполнения при заполнености не более 10%. Для матриц 100-400 элементов CSR/CSC требует меньше памяти при заполнености не более 50%. Матрицы с 1.000 элементов при использовании CSR/CSC требуют меньше памяти при заполнености не более 30%. Для матриц с 4.000 элементами при помощи CSR/CSC можно экономить памяти при заполнености не более 75%. Для матриц 5.000-10.000 экономия по памяти при заполнености не более 50%. Для матриц 500.000 элементов CSR/CSC позволяет экономить память при заполнености не более 30%

По результат анализа, полученных данных можно сказать, что метод хранения CSR/CSC использовать на матрицах с количество элементов меньше 50 не эффективно. Для матриц с количество элементов больше 50, важно учитывать процент заполененности матрицы. По полученным данным, что можно указать, что для матриц с количеством элементов от 50 до 500.000 при проценте заполненности не более 30% для экономии памяти можно использовать метод хранения CSR/CSC. Но при этом если процент заполненности больше 10%, то необходимо больше времени на выполнение умножения.

Метод CSR/CSC имеет наибольшую эффективность при заполненности матрицы не более 10%, при данных значениях мы получаем преимущество и по времени выполнения и по памяти. При заполненности не более 30% метод всё также остается эффективным по использованной памяти, но требует больше времени на выполнение умножения. При заполненности не более 50% эффективен только по памяти для матриц с 100-400 и 5.000-10.000 элементами. При заполненности не более 75% эффективен только по памяти для матриц с 4.000 элементами.

Контрольные вопросы

1. Что такое разреженная матрица, какие схемы хранения таких матриц Вы знаете?

Разряженная матрица – матрица, в которой значительная часть элементов является 0.
Способы хранения:

- CSR
- CSC
- Координатное хранение
- Ленточные
- Диагональные

- Профильная
- Вперед по строке, назад по столбцу

2. Каким образом и сколько памяти выделяется под хранение разреженной и обычной матрицы?

Выделение памяти

- Для разряженной матрицы память выделяется динамически
- Для обычной матрицы память выделяется постоянный объем

Количество памяти

- Для разряженной матрицы зависит от количества ненулевых элементов
- Для обычной матрицы объем равен $n * m * \text{sizeof}(\text{matrix_type})$

3. Каков принцип обработки разреженной матрицы?

Основной принцип обработки разряженной исключение из обработки нулевые элементы, в отличии от обработки обычной матрицы, в которой обрабатывается каждый элемент.

4. В каком случае для матриц эффективнее применять стандартные алгоритмы обработки матриц? От чего это зависит?

Стандартные алгоритмы обработки матриц эффективнее для матриц, которые заполнены на более чем 30% или содержат менее 50 элементов, то есть при малом количестве нулевых элементов или маленьких размерах матрицы.